산업 IoT 통신 실습 · Python 기반

MQTT 4단계 실습

Pub/Sub · 토픽 와일드카드 · Retained · LWT 동작 검증

broker.emqx.io:1883 · paho-mqtt

STEP 1
Pub/Sub

STEP 2
토픽 와일드카드

STEP 3
Retained Message

STEP 4
LWT

실습 준비

실습 개요 및 개발 환경 구성

별도 브로커 구축 없이 로컬 Python 환경과 공개 테스트 브로커를 이용해 실습을 수행한다.

터미널 1 · Subscriber (관제)

```
$ python sub_console.py
```

터미널 2 · Publisher (센서)

```
$ python pub_step1_basic.py
```

환경 준비 및 패키지 설치

필요 환경 Python 3.8 이상, 터미널 2개 (CMD / PowerShell / Bash)

```
$ pip install paho-mqtt
```

공개 테스트 브로커 broker.emqx.io · 기본 포트 1883

외부에 공개된 테스트용 브로커로, 별도 설치 없이 접속할 수 있다. 단, 네트워크에서 1883 포트의 외부 접속이 허용되어야 한다.

필수 · USER_ID 기반 토픽 분리

공개 브로커는 모든 사용자가 하나의 토픽 공간을 공유한다. 토픽 충돌을 방지하기 위해 모든 토픽 앞에 본인 고유 ID(이니셜 또는 학번)를 접두어로 사용한다.

토픽 명명 규칙

```
[USER_ID]/factory/line1/...
예) hjkim/factory/line1/press01/temp
```

STEP 1 · Pub/Sub

단일 Publisher–Subscriber 간 메시지 전달

Publisher와 Subscriber는 브로커 주소와 토픽만 공유하며, 발행된 메시지는 브로커를 거쳐 구독자에게 전달된다.

터미널 2 · 센서
Publisher

역할 1호기 프레스의 온도 센서

1초 주기로 가상 온도 데이터(25.3°C, 25.8°C ...)를 JSON 형식으로 발행

```
$ python pub_step1_basic.py
```

Publish
25.3°C

중앙 브로커

`broker.emqx.io:1883`

토픽 필터가 일치하는 구독자에게 메시지를 전달(push)

Push

터미널 1 · 관제
Subscriber

역할 공장 중앙 관제 대시보드

구독 후 on_message 콜백으로 수신 메시지 처리

```
$ python sub_console.py
```

구독 · 발행 토픽

`[USER_ID]/factory/line1/press01/temp`

확인 사항

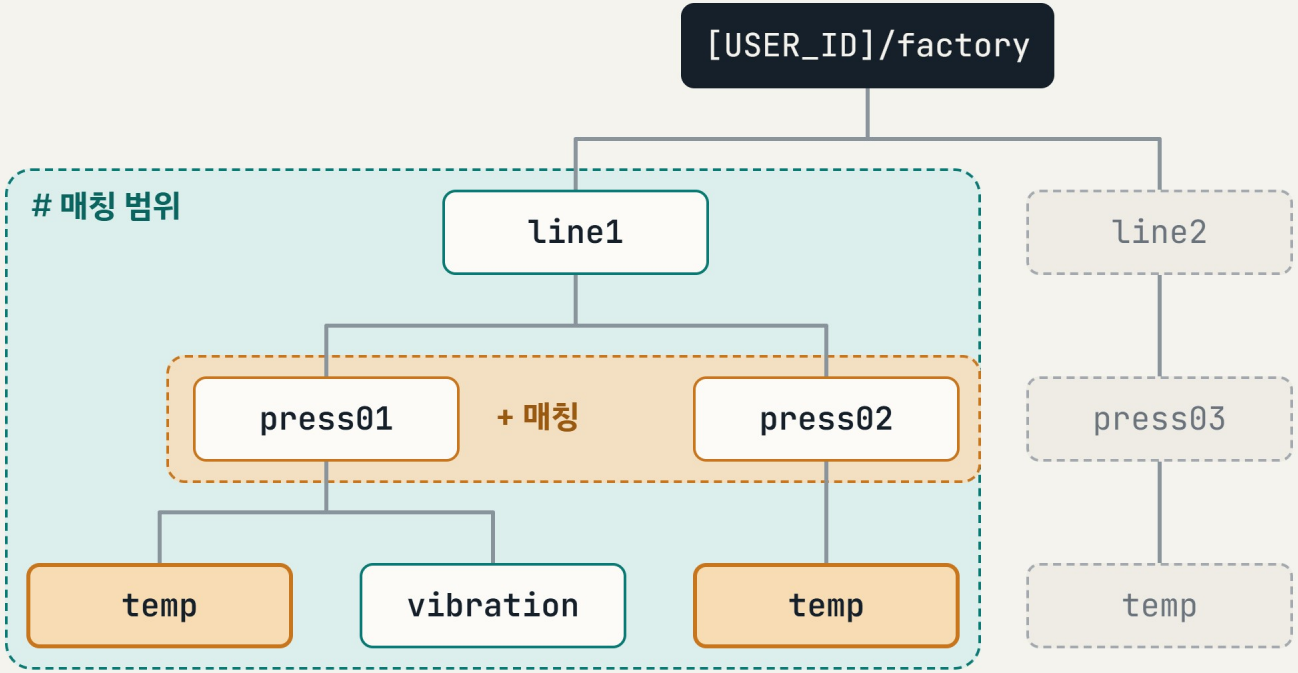
발행과 동시에 구독 터미널에 메시지가 출력되는지 확인 (폴링 없는 이벤트 기반 수신)

STEP 2 · 와일드카드

토픽 계층 구조와 와일드카드(+, #) 필터링

와일드카드(+, #)를 사용하면 하나의 토픽 필터로 다수 설비의 데이터를 선택적으로 구독할 수 있다.

발행 토픽 press01/temp · press02/temp · press01/vibration (비교군: line2/press03/temp)



Subscriber 1 · 단일 레벨 와일드카드 +

`[USER_ID]/factory/line1/+/temp`

press01·press02의 temp 수신, vibration은 토픽 필터와 불일치하여 미수신

Subscriber 2 · 다중 레벨 와일드카드 #

`[USER_ID]/factory/line1/#`

line1 하위의 모든 토픽(press01·press02, temp·vibration) 수신

STEP 3 · Retained Message

Retained Message: 최신 상태 보존과 신규 구독자 동기화

Retained 메시지는 브로커에 보관되어, 이후 구독한 클라이언트에게 즉시 전달된다.

① 일반 메시지 · Retained = False

센서가 상태(RUNNING)를 발행한 후 종료

∨

10초 후 관제 콘솔(Subscriber) 실행

∨

```
[관제 콘솔] 구독 시작
… (데이터 없음)
```

결과: 다음 발행 시점까지 수신 메시지 없음

② Retained 메시지 · Retained = True

센서가 retain=True로 최신 상태 1건을 발행한 후 종료

∨

브로커 보관 메시지 · RETAINED

`{"status": "RUNNING"}`

토픽당 마지막 메시지 1건 보관

∨

```
[RETAINED] 토픽: …/press01/state
└ 페이로드: {"status": "RUNNING"}
```

결과: 신규 구독자가 구독 직후 최신 상태 수신

STEP 4 · Last Will and Testament

LWT: 비정상 연결 종료 감지

클라이언트가 DISCONNECT 없이 연결을 잃으면, 브로커가 사전 등록된 Will 메시지를 대신 발행한다.

센서 press01

Publisher · keepalive=5

① Will 메시지 사전 등록

connect() 이전에 will_set()으로 토픽과 페이로드 지정 (CONNECT 패킷에 포함)

```
…/press01/status
"status": "CRITICAL_OFFLINE"
```

② 비정상 연결 종료

실행 중 Ctrl + C 또는 kill로 프로세스 강제 종료 → DISCONNECT 패킷 미전송

→

브로커

③ 브로커의 Will 메시지 대리 발행

Will 메시지 발행

`status → CRITICAL_OFFLINE`

DISCONNECT 없는 연결 종료를 감지하면, 보관 중인 Will 메시지를 해당 토픽의 구독자에게 발행

프로세스 강제 종료: 소켓 종료 즉시
전원·네트워크 단절: Keep-Alive × 1.5 경과 후

→

⚠ 터미널 1 · 관제

```
토픽: …/press01/status
└ "CRITICAL_OFFLINE"
```

현장 확인 없이 원격 설비의 전원 차단·통신 두절을 인지

STEP 4 · 확인 실험

Will 메시지의 등록·보관 확인

Publisher 코드에는 OFFLINE 발행 구문이 없다. 관제 콘솔에 OFFLINE이 수신되면 브로커가 보관하던 Will 메시지를 발행한 것이다.

① 실행 직후

pub_step4_lwt.py 실행 → Will 메시지 등록 후 status에 ONLINE 발행

```
토픽: …/press01/status
└ "status": "ONLINE"
```

센서가 직접 발행한 정상 상태

② 강제 종료 직후

5초 후 os._exit(1)로 자동 종료 → Publisher 프로세스 소멸

```
토픽: …/press01/status
└ "CRITICAL_OFFLINE"
```

발행 주체가 없는 상태에서 수신 → 브로커의 대리 발행

③ 관제 콘솔 재실행

터미널 1에서 Ctrl + C 후 sub_console.py 재실행

```
[RETAINED] …/press01/status
└ "CRITICAL_OFFLINE"
```

Will 메시지가 브로커에 보관되어 ONLINE을 대체

② **에서 [RETAINED] 표시가 없는 이유** 이미 구독 중인 클라이언트에게는 retain 플래그를 0으로 전달하기 때문이다(MQTT 3.1.1). 새로 구독하는 ③ 에서만 [RETAINED]로 표시된다.

정리 · 트러블슈팅

4단계 핵심 정리 및 트러블슈팅

네 가지 메커니즘은 통신이 불안정한 산업 현장에서 무선 통신의 확장성과 신뢰성을 보완한다.

STEP 1

Pub/Sub

송·수신 측의 직접 결합을 제거하여 설비 증설과 교체가 용이

STEP 2

와일드카드

계층형 토픽 설계로 전체 모니터링과 개별 설비 구독을 함께 지원

STEP 3

Retained

구독자 재접속·재시작 시 설비의 최신 상태를 즉시 제공

STEP 4

LWT

원격 설비의 전원 차단·통신 두절을 브로커 단에서 자동 감지

자주 발생하는 실습 오류 (FAQ)

Q. 메시지가 수신되지 않음	Subscriber와 Publisher의 USER_ID가 정확히 일치하는지 확인
Q. 브로커 연결 실패	기관 네트워크에서 1883 포트가 차단되었을 가능성 → 모바일 핫스팟 등 외부망으로 전환
Q. 재실행 시 이전 상태가 수신됨	Step 3·4의 Retained 메시지가 브로커에 남아 있음 → 같은 토픽에 빈 페이로드를 retain=True로 발행하여 삭제

CODE · 파일 구성

실습 파일 구성과 실행 순서

파일	실행 위치	역할	확인 포인트
sub_console.py	터미널 1 (상시)	통합 관제 수신기 (factory/#)	모든 패킷, [RETAINED] 표시
pub_step1_basic.py	터미널 2	1초 주기 온도 발행	실시간 수신
pub_step2_wildcard.py	터미널 2	설비·센서 4개 토픽 발행	계층별 수신 범위
pub_step3_retained.py	터미널 2	Retained 상태 발행 후 종료	콘솔 재실행 시 즉시 수신
pub_step4_lwt.py	터미널 2	LWT 등록 후 강제 종료	ONLINE → OFFLINE 전환

USER_ID 통일

5개 파일의 USER_ID = "myname"을 동일한 본인 ID로 수정

paho-mqtt v2.0+ / v1.x 호환 (우측 함수)

```
def get_mqtt_client():
    try:
        # paho-mqtt v2.0 이상
        return mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    except AttributeError:
        # paho-mqtt v1.x 구버전
        return mqtt.Client()
```

CODE · 터미널 1

sub_console.py — 통합 관제 수신기

```
USER_ID = "myname"  # 본인 이니셜/학번으로 변경
BROKER = "broker.emqx.io"
TOPIC_ALL = f"{USER_ID}/factory/#"

def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        client.subscribe(TOPIC_ALL)

def on_message(client, userdata, msg):
    retained_flag = "[RETAINED] " if msg.retain else ""
    print(f"[{timestamp}] {retained_flag}토픽: {topic}")

client = get_mqtt_client()
client.on_connect = on_connect
client.on_message = on_message
client.connect(BROKER, PORT, keepalive=60)
client.loop_forever()
```

상시 실행

실습 전 과정에서 터미널 1에 실행해 두는 중앙 관제 콘솔

factory/# 구독

본인 ID 하위의 모든 토픽 수신 (line2 포함)

[RETAINED] 표시

msg.retain으로 보관 메시지를 구분 — Step 3·4 확인용

페이로드 형식 처리

JSON으로 파싱하여 출력하고, 파싱 실패 시 문자열로 출력

CODE · STEP 1 · 터미널 2

pub_step1_basic.py — 주기적 온도 발행기

```
TOPIC = f"{USER_ID}/factory/line1/press01/temp"

client = get_mqtt_client()
client.connect(BROKER, PORT, keepalive=60)
client.loop_start()

count = 1
while True:
    current_temp = round(random.uniform(25.0, 28.0), 2)
    payload = {"sensor_id": "press01_temp", "sequence": count,
               "temperature": current_temp, "unit": "Celsius"}
    # QoS 0 등급으로 발행
    client.publish(TOPIC, json.dumps(payload), qos=0)
    count += 1
    time.sleep(1)
```

단일 토픽

1호기 프레스 온도 press01/temp

1초 주기 JSON

25.0~28.0°C 가상 온도, sequence 필드로 발행 순번 표시

QoS 0

수신 확인(ACK) 없이 최대 1회 전달 (at most once)

정상 종료

Ctrl + C → loop_stop(), disconnect() 호출 — Step 4와 비교

CODE · STEP 2 · 터미널 2

pub_step2_wildcard.py — 와일드카드 검증 발행기

```
data_samples = [
    (f"{USER_ID}/factory/line1/press01/temp",
     {"machine": "press01", "temp": 26.5}),
    (f"{USER_ID}/factory/line1/press02/temp",
     {"machine": "press02", "temp": 27.8}),
    (f"{USER_ID}/factory/line1/press01/vibration",
     {"machine": "press01", "vibe_rms": 0.42}),
    (f"{USER_ID}/factory/line2/press03/temp",
     {"machine": "press03", "temp": 24.1})
]

for topic, payload in data_samples:
    client.publish(topic, json.dumps(payload), qos=0)
    print(f"발행 완료 ──> 토픽: {topic}")
    time.sleep(0.5)
```

4개 토픽 1회 발행

line1 3건 + line2 1건(비교군), 0.5초 간격

+ 검증

line1/+/temp 구독 → press01·press02 온도만 수신

검증

line1/# 구독 → line1 3건 수신, line2 제외

주의

기본 sub_console.py(factory/#)는 4건 모두 수신 → TOPIC_ALL 변경 후 비교

CODE · STEP 3 · 터미널 2

pub_step3_retained.py — Retained 상태 발행기

```
TOPIC_STATE = f"{USER_ID}/factory/line1/press01/state"

state_payload = {
    "equipment": "press01",
    "status": "RUNNING",
    "operator": "Kim_HJ",
    "timestamp": time.strftime("%Y-%m-%d %H:%M:%S")
}

# retain=True: 브로커가 이 메시지를 메모리에 보관
client.publish(TOPIC_STATE, json.dumps(state_payload),
               qos=1, retain=True)

time.sleep(1)
client.loop_stop()
client.disconnect()
```

retain=True · QoS 1

브로커가 토픽의 마지막 상태 1건을 보관

발행 후 즉시 종료

Publisher 종료 후에도 상태 메시지는 브로커에 유지

실습 확인

관제 콘솔 재실행 시 [RETAINED] 표시와 함께 즉시 수신

정리

같은 토픽에 빈 페이로드를 retain=True로 발행하여 삭제

CODE · STEP 4 · 터미널 2

pub_step4_lwt.py — LWT 등록과 비정상 종료 재현

```
lwt_payload = json.dumps({
    "equipment": "press01",
    "status": "CRITICAL_OFFLINE",
    "reason": "Sudden Power Cut / Network Lost"
})
# [핵심] 브로커 연결 전에 Will 메시지(LWT) 등록
client.will_set(TOPIC_STATUS, payload=lwt_payload,
                qos=1, retain=True)
client.connect(BROKER, PORT, keepalive=5)
client.loop_start()
# 접속 직후 정상 상태(ONLINE) 발행
client.publish(TOPIC_STATUS, online_payload, qos=1, retain=True)
for i in range(1, 6):
    data = {"power_kw": 45.2, "count": i}
    client.publish(TOPIC_TELEMETRY, json.dumps(data))
    time.sleep(1)
os._exit(1)  # disconnect() 없이 프로세스 강제 종료
```

will_set()은 connect() 전

Will 메시지는 CONNECT 패킷에 실려 브로커에 등록됨

keepalive=5

Keep-Alive 주기를 짧게 설정하여 단절 감지 시간 단축

os._exit(1)

DISCONNECT 패킷 없이 종료 → 브로커가 Will 메시지 대리 발행

ONLINE 발행 (retain)

status의 보관 메시지가 ONLINE → Will 메시지(OFFLINE)로 교체되는지 확인